

T1 Track Representation — Code Guide

Every script, class and important function of `t1_track/`, with the numbers from the test runs.

Companion to `docs/T1_ASSUMPTIONS.md` and `planning_task_breakdown.md`. Tip: Ctrl+P → "Save as PDF" for a printable copy.

0. The big picture

T1 = Track representation. During one slow test lap, perception records what the car sees. T1 turns that recording into a map of the track that the later tasks (T2 → T8) use.

Input

log.jsonl: one JSON line per frame (t, pose, velocity, boundary_points, obstacles). Format from docs/T1_ASSUMPTIONS.md §3.

Output

track.csv (s, x, y, w_left, w_right, one row every 5 cm) + track_meta.json (what, where, how, and whether it passed the checks).

One frame (first line of the synthetic log)

t:	0.0	time [s]
pose:	x = 6.3, y = 0.0, yaw = 1.35	where the car is + which way it faces [m, m, rad]
velocity:	vx = 2.0, vy = 0.0	not used by T1 (carried along)
boundary_points:	73 points, e.g. [0.21, 0.87]	wall points the LiDAR saw right now
obstacles:	[]	ignored by T1

With --points-frame vehicle, [0.21, 0.87] means "0.21 m ahead of the car and 0.87 m to its left".

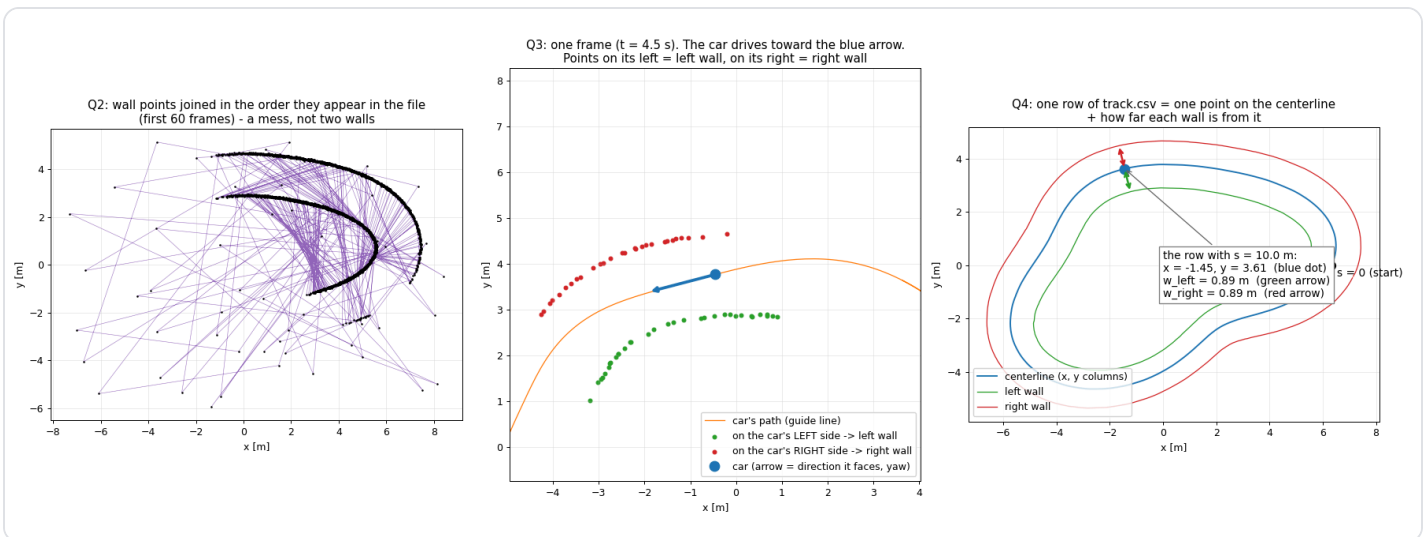
One row of track.csv

s,	x,	y,	w_left,	w_right
10.0,	-1.45,	3.61,	0.89,	0.89

"10 m along the track (driving direction), the middle of the track is at (−1.45, 3.61), the left wall is 0.89 m away, the right wall is 0.89 m away." All rows together describe the whole track; the walls are rebuilt as *centerline ± width × left-direction*.

The problem and the key idea

All frames together give a big cloud of wall points, but it is **unordered** (no "next point"), **unlabelled** (no left/right) and **noisy**. The key idea: the car's own path (its poses) is already an ordered loop, in the driving direction, between the two walls. It is used as a **guide line**: order = follow it; left/right = which side of it a point is on; middle = measure both walls and shift to halfway.



Q2: points in file order are a tangle, the order must be created. Q3: left/right = which side of the car's direction a point is on. Q4: one row of track.csv.

1. The pipeline and the files

log.jsonl (375 lines)

▼ log_io.py	read each line -> Frame (t, pose, points ...)	375 Frames
▼ accumulate.py	all frames -> ONE clean cloud of wall points + the car's path (poses)	26,040 -> 3,173 points
▼ extract.py	cut one lap -> guide line -> left/right split -> widths -> shift to the middle (2 passes) (uses the maths tools in geometry.py)	647 points every 5 cm
▼ validate.py	10 checks: errors (block) / warnings (flag)	valid: true
▼ export.py	write track.csv + track_meta.json	

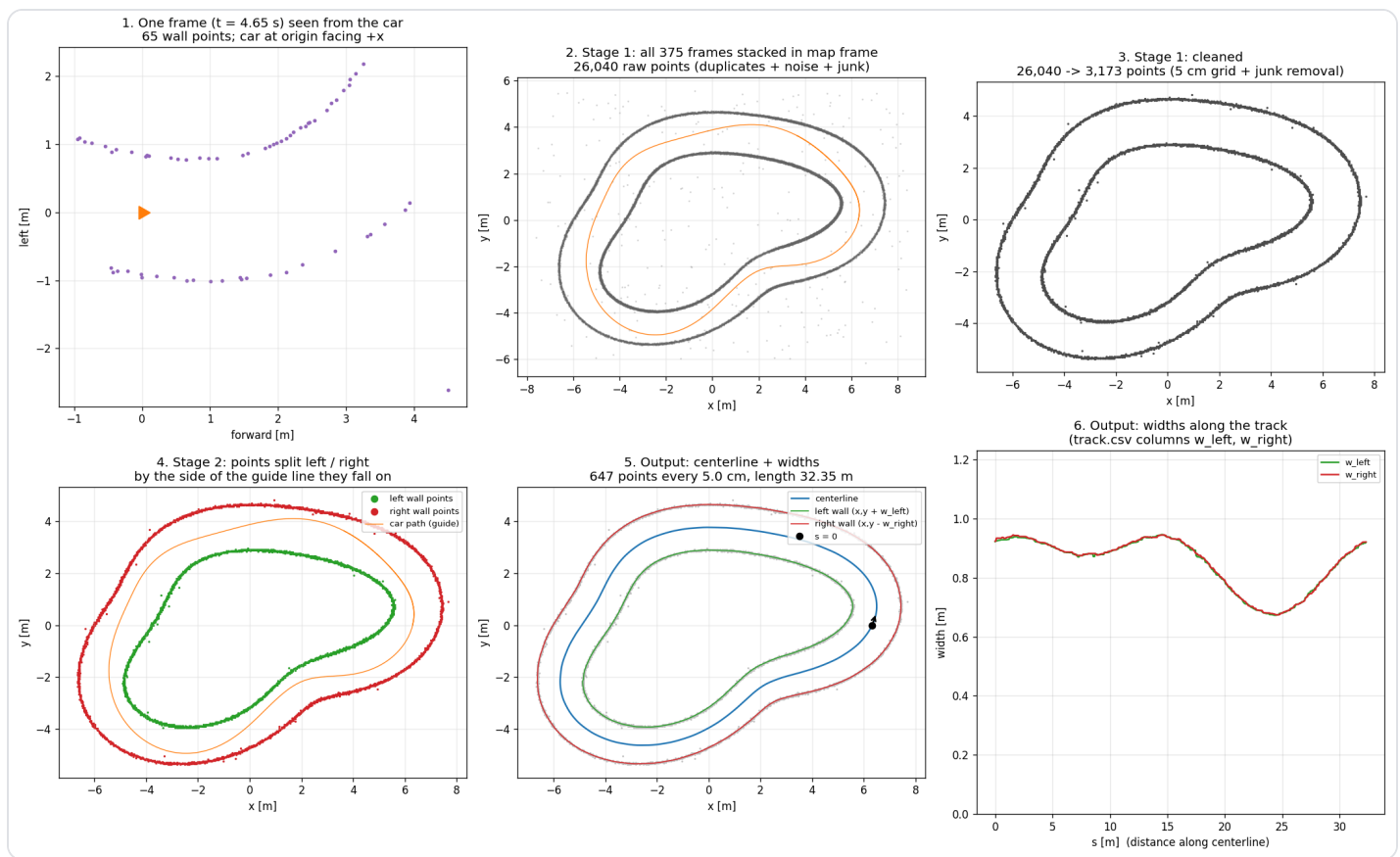
config.py = every setting (one box of numbers)

pipeline.py = build_track(): runs the steps above in order

__main__.py = the command line: python -m t1_track build ...

File	Job	Main things inside	Lines (approx.)
config.py	all settings in one place	T1Config	55
log_io.py	text file → Python objects	Frame, parse_frame, read_log	80
accumulate.py	Stage 1: frames → one clean point cloud	Accumulated, to_world, voxel_downsample, remove_isolated, accumulate	70
geometry.py	toolbox of maths for closed lines	closed_arclength, dedupe, resample_closed, smooth_closed, tangents_normals, self_intersections	85
extract.py	Stage 2: cloud + path → centerline + widths	Track, extract_lap, build_reference, lateral_profile, extract_track	155
validate.py	is the result trustworthy?	validate, is_valid, warnings	100
export.py	write the deliverables	write_track_csv, build_meta, write_outputs	70
pipeline.py	glue: runs everything	build_track, BuildResult	50
__main__.py	command line	synth, build, realtrack commands	130

Testing tools (not part of the pipeline): `synth.py` (fake track + log), `realtrack.py` (log from a real map), `plot.py` (pictures), `tools/gym_record.py` (log from the official F1TENTH simulator), `tests/` (automatic tests).



The whole pipeline on the synthetic log: 1 one frame, 2 all frames stacked, 3 cleaned, 4 left/right split, 5 output, 6 widths along s.

2. config.py — the settings box

Does nothing by itself. One `@dataclass` (`T1Config`) holding every number, so no other file hides numbers. `T1Config()` = all defaults; `T1Config(points_frame="vehicle", ds=0.10)` = change some, keep the rest. `__post_init__` runs automatically and rejects bad values (e.g. a typo "vehicel", or $ds \leq 0$).

Setting	Default	Meaning	Used in
points_frame	"world"	boundary points already in map coordinates ("world") or relative to the car ("vehicle"). Unconfirmed with perception.	accumulate
max_point_range	10.0 m	ignore points farther than this from the car (1° heading error moves a 30 m point 52 cm)	accumulate
voxel_size	0.05 m	grid square size for merging duplicates	accumulate
min_hits_per_voxel	1	drop squares with fewer points (1 = keep all)	accumulate
outlier_radius / outlier_min_neighbors	0.25 m / 3	a point needs ≥ 3 neighbours within 25 cm, else it is junk	accumulate
closure_radius	0.5 m	how close the car must come back to its start to close the lap	extract_lap
min_lap_length	5.0 m	must drive this far before a return counts as a lap	extract_lap
ds	0.05 m	spacing of the output centerline (breakdown: ~5 cm)	extract
ref_smoothing	0.5 m	smoothing of the car's path into the guide line	build_reference
max_half_width	3.0 m	ignore wall points farther than this from the line	lateral_profile
station_window	0.3 m	each station also uses points within ± 0.3 m along the line	lateral_profile
width_percentile	50	statistic of the offsets per station (50 = median)	lateral_profile
width_median_window	0.25 m	median filter on widths: removes spikes only	lateral_profile
destaircase	0.10 m	very light smoothing of the centerline (real smoothing = T2)	extract_track
iterations	2	number of re-centring passes	extract_track
frame_id	"map"	frame name written to the metadata	export

Setting	Default	Meaning	Used in
min_width	0.2 m	smallest acceptable width / clearance	validate
max_gap_fraction	0.2	at most 20% of a wall may be unseen	validate
max_length_mismatch	0.10	centerline length within 10% of the driven lap	validate
max_spacing_deviation	0.05	spacing within 5% of ds	validate

3. log_io.py — reading the log

Input

path to log.jsonl

Output

(frames, skipped): list of Frame objects
sorted by time + number of broken lines
skipped

```
@dataclass
class Frame:
    t: float                # time
    pose: np.ndarray         # [x, y, yaw]
    velocity: np.ndarray     # [vx, vy]   (unused by T1)
    boundary_points: np.ndarray # N rows x 2 columns (x, y)
    obstacles: list[dict]    # unused by T1
```

Function	In → Out	What it does
parse_frame(obj)	dict → Frame	builds one Frame. Bad pose (missing / NaN) → the whole frame is rejected (its points could not be placed). Bad point (NaN) → only that point is dropped. .reshape(-1, 2) forces N×2 (also works for 0 points). Missing velocity → NaN.
read_log(path)	file → (frames, skipped)	line by line: json.loads (text → dict) then parse_frame (dict → Frame). A broken line is skipped, counted and reported (file:line + reason) instead of crashing. Empty lines ignored. No valid frame at all → error. Frames sorted by t.
frame_to_dict, write_log	Frame → JSON line	the reverse; only the fake-data generators use them.

Error caught in read_log	Example of a bad line
JSONDecodeError	cut-off line {"t": 1.0, "pose":
KeyError	a required field missing, e.g. no "pose"
TypeError	wrong type, e.g. "pose": 5
ValueError	NaN in the pose

Why 375 frames? Only because of the synthetic generator's settings: frames = laps × lap length / (speed × dt) = $1.15 \times 32.61 \text{ m} / (2.0 \text{ m/s} \times 0.05 \text{ s}) = 375$. With the real car: frames = lap time × logging rate (e.g. $40 \text{ s} \times 10 \text{ Hz} = 400$). The extra 0.15 lap guarantees the car passes its start again (needed to close the lap).

4. accumulate.py — Stage 1: one clean point cloud

Input

list of Frames + config

Output

Accumulated: t (375), poses (375 × 3, the car's path), points (3,173 × 2, clean wall points on the map), n_raw_points

After this file the points are clean but still unordered and unlabelled (no left/right yet: that is extract.py's job).

```
accumulate(frames, cfg):
    poses = all frame poses                    -> the car's path
    for each frame:
        pts = to_world(points, pose)  (only if points_frame == "vehicle")
        keep points <= 10 m from the car      (max_point_range)
    raw = stack all frames                    26,040 -> 25,987 points
    voxel_downsample (5 cm grid, average per square) -> 3,361
    remove_isolated (< 3 neighbours within 25 cm) -> 3,173
```

to_world(points, pose) — car's view → map

```
x, y, yaw = pose
c, s = cos(yaw), sin(yaw)
return points @ [[c, s], [-s, c]] + (x, y)    # rotate by yaw, then shift by (x, y)

# per point (px, py):  map_x = x + cos(yaw)*px - sin(yaw)*py
#                      map_y = y + sin(yaw)*px + cos(yaw)*py
```

Worked example: car at (5, 2) facing north (yaw = 90°, cos = 0, sin = 1), wall 1 m to its left = (0, 1) → map_x = 5 - 1 = 4, map_y = 2 → **(4, 2)**: 1 m west of the car, correct (facing north, left = west).

voxel_downsample(points, cell) — merge duplicates

The same wall is seen in dozens of frames (≈ 1,000 points on a 90 cm patch). A 5 cm grid is laid over the map; all points in one square are replaced by their average. Removes repetition and cancels noise.

```

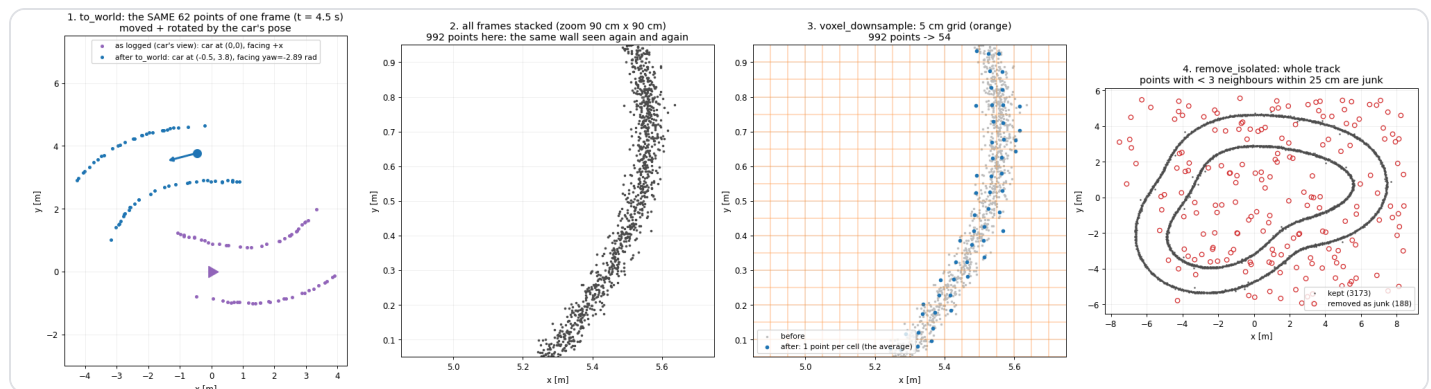
keys = floor(points / cell)                                # grid square number of each point, e.g. (0.03, 0.02) -
_, inv, counts = np.unique(keys, ...)                      # which square each point is in, how many per square
np.add.at(sums, inv, points)                               # add every point into its square's total
centroids = sums / counts                                  # average position per square

```

remove_isolated(points, radius, min_neighbors) — remove junk

Real walls are dense; junk (noise, reflections) is alone. cKDTree = fast neighbour search.

query_ball_point(..., return_length=True) - 1 = neighbours within 25 cm (minus the point itself). Keep points with ≥ 3 .



1 to_world: same points, moved + rotated by the pose. 2 all frames stacked: 992 points on 90 cm of wall. 3 voxel grid: 992 → 54. 4 junk removed (red circles).

5. geometry.py — the maths toolbox

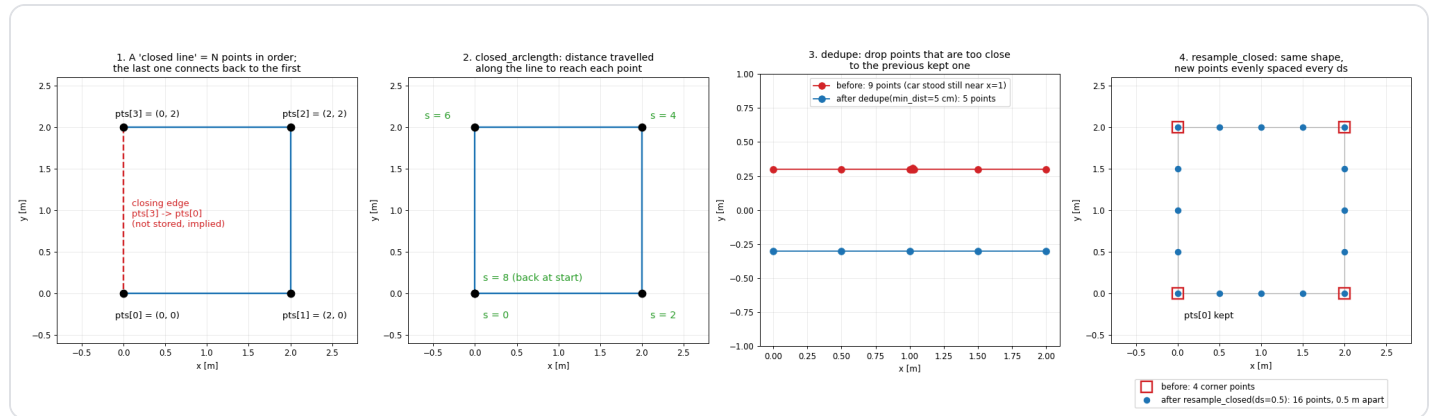
Not a pipeline step: small tools called by extract.py and validate.py. Everything is about **closed lines**: N points in order, **the last connects back to the first** (the first point is NOT repeated). That is why several functions do `closed = np.vstack([pts, pts[:1]])` to make the closing edge explicit.

Tool	In → Out	What / how	Used by
<code>closed_arclength(pts)</code>	N points → N+1 numbers	distance walked along the line to each point (+ back to start). <code>diff</code> → <code>norm</code> → <code>cumsum</code> . Square 2 m: [0, 2, 4, 6, 8]; last = total length.	the s column, test-lap length, resample
<code>dedupe(pts, min_dist)</code>	points → fewer points	drops points closer than <code>min_dist</code> to the last kept point (car standing still).	<code>build_reference</code>
<code>resample_closed(pts, ds)</code>	closed line → same shape, evenly spaced	<code>n = round(length / ds)</code> points at <code>s = 0, L/n, 2L/n ...</code> ; <code>np.interp</code> finds x, y at each s. <code>pts[0]</code> is kept. If <code>ds</code> does not divide L, the closest spacing that does is used.	guide line, centerline (every 5 cm)
<code>smooth_closed(v, sigma)</code>	values → smoothed values	Gaussian weighted average of neighbours. <code>mode='wrap'</code> = last point's neighbour is the first → no seam at start/finish. <code>sigma</code> is in points : metres / <code>ds</code> .	guide line (0.5 m), de-staircase (0.10 m)
<code>tangents_normals(pts)</code>	points → (t, n)	<code>t</code> = next – previous, length 1 = forward . <code>n</code> = <code>t</code> turned 90° left: (<code>tx</code> , <code>ty</code>) → (<code>-ty</code> , <code>tx</code>) = left .	left/right split, widths, walls, re-centring
<code>self_intersections(pts) / is_simple_closed</code>	line → crossing edge pairs / True-False	checks only nearby edge pairs (cKDTree), skips neighbouring edges, orientation test: AB and CD cross if C, D are on opposite sides of AB and A, B on opposite sides of CD.	validate

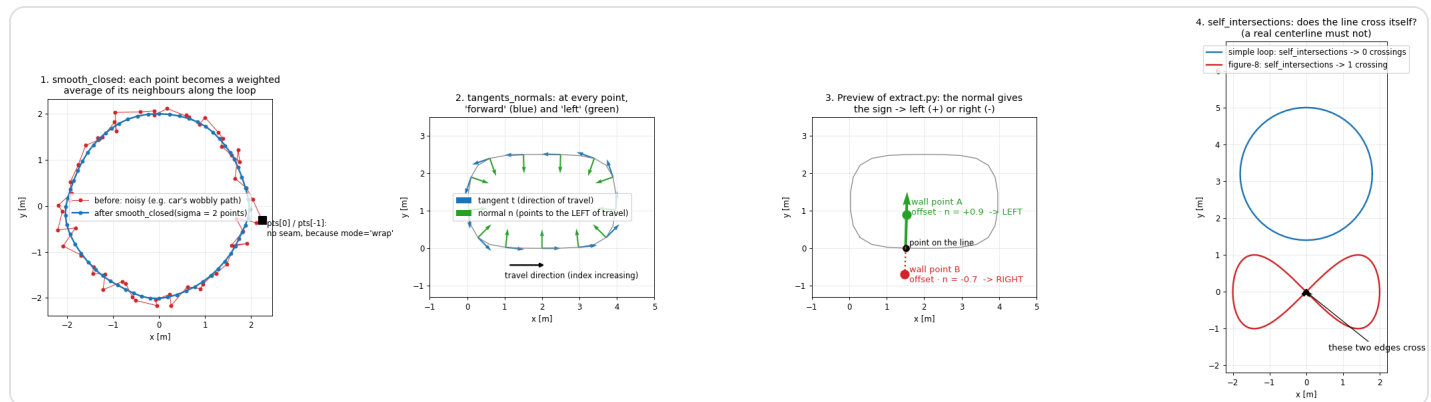
The left/right sign (the core of T1)

```
offset = (wall_point - point_on_line) @ n      # dot product with the LEFT direction
# offset > 0 -> LEFT wall, offset < 0 -> RIGHT wall, |offset| = sideways distance
```

"Left" only exists relative to a direction of travel: drive the same loop the other way and the normals flip.



Closed line, closed_arclength, dedupe, resample_closed on a 2 m square.



smooth_closed, tangents (forward) and normals (left), the left/right sign, self_intersections.

6. extract.py — Stage 2: centerline + widths

Input

poses (car's path) + points (clean cloud) +
config

Output

Track: s, xy, w_left, w_right, length, ds, stats
(= content of track.csv + part of the metadata)

```
extract_track(poses, points, cfg):
    lap = extract_lap(poses)           ① exactly one lap of the car's path      375 -> 326 pos
    ref = build_reference(lap)          ② clean guide line, point every 5 cm    646 points
    repeat 2 times:                    ③ re-centring
        prof = lateral_profile(ref, points)    left/right split + both widths
        ref = ref + n * (w_left - w_right)/2    move every point to the middle
        ref = resample(light_smooth(ref))      tidy, exactly 5 cm
    prof = lateral_profile(ref, points)  ④ final widths
    return Track(...)                  ⑤ s from closed_arclength, length, ds, stats
```

Classes: `ExtractionError` (data cannot make a track, e.g. incomplete lap → CLI exit code 2),
`WidthProfile` (w_left, w_right, gap fractions, points used), `Track` (the result).

① `extract_lap(xy, closure_radius, min_lap_length)`

```
cum = distance driven so far at each pose
d0 = straight distance from each pose back to the START
candidate = (cum >= 5 m) and (d0 <= 0.5 m)      # driven far enough AND back near the start
none? -> ExtractionError("never returns ... is the test lap complete?")
end = among the first group of candidates, the pose CLOSEST to the start
return xy[:end], d0[end]                        # closing pose excluded (first point not repeated)
```

Synthetic log: end = pose #326, 0.9 cm from the start; lap = 326 poses, 32.61 m; 49 extra poses dropped. Both conditions are needed: without " ≥ 5 m", pose 0 and 1 would count (the car is still at its start).

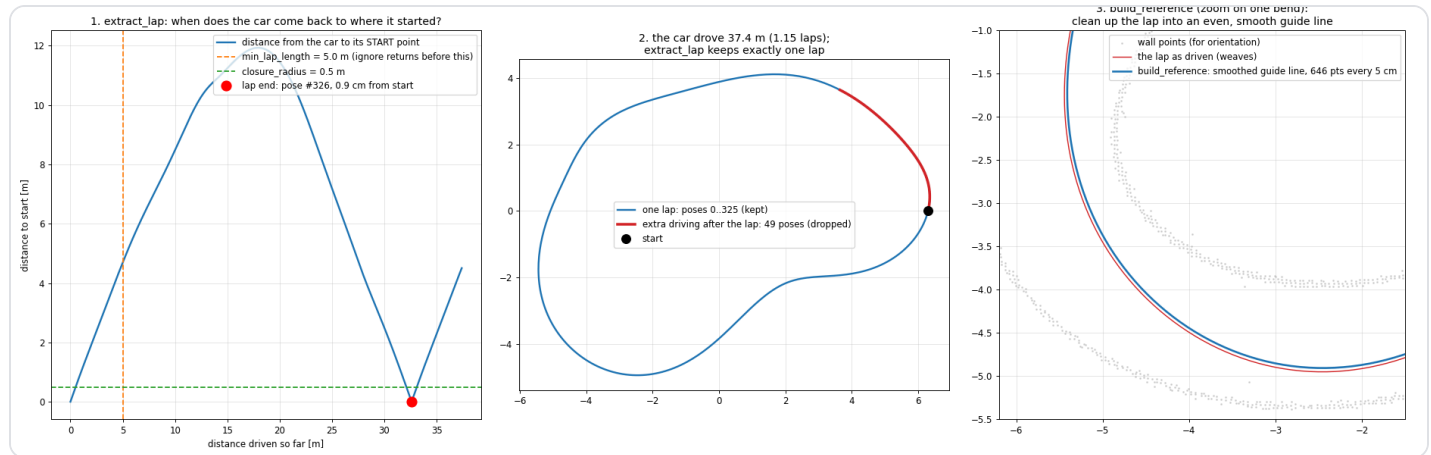
② `build_reference(lap)`

```

ref = resample_closed(dedupe(lap, ds/2), ds)      # drop poses < 2.5 cm apart, then every 5 cm  ->
ref = smooth_closed(ref, ref_smoothing / ds)      # 0.5 m / 0.05 m = sigma 10 points
ref = resample_closed(ref, ds)                   # smoothing shortens the line -> re-space

```

The guide line is **not** in the middle (it follows the car's weave). That is fine: it only has to be ordered, in the driving direction, and between the walls.



1 when the car comes back to its start (red dot = lap end), 2 one lap kept / extra dropped, 3 the guide line.

③ lateral_profile(ref, points) — 4 steps

```

# 1. left/right split
normals = tangents_normals(ref)
dist, idx = cKDTree(ref).query(points)          # nearest line point ("station") for each wall
lat = einsum(points - ref[idx], normals[idx])    # dot product: + left, - right, size = distance
keep = dist <= 3 m                             # max_half_width

# 2. put each |offset| in the bucket of its station (sort + searchsorted), left and right separately

# 3. one width per station
for station i: take the buckets of stations i-6 ... i+6   (station_window 0.3 m / 0.05 m, % n wraps
    w[i] = median(offsets)                                (NaN if no points)

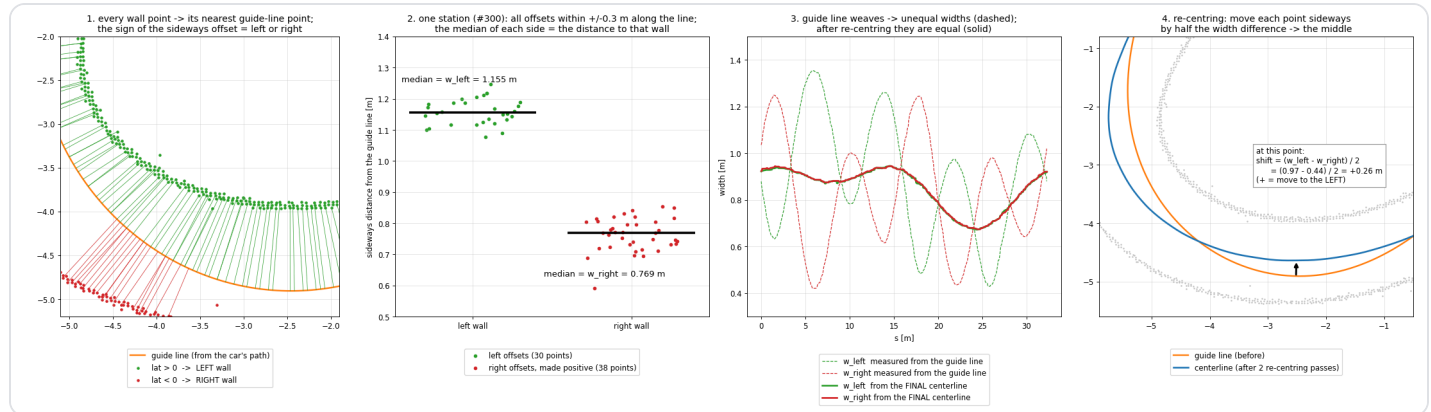
# 4. gap = fraction of NaN stations (100% -> ExtractionError)
#   _fill_circular: fill NaN by straight line between known neighbours (wraps)
#   despike_closed: median over 5 stations (0.25 m), removes single spikes

```

Synthetic log: 1,390 points left, 1,783 right. **Why the median:** offsets 0.90, 0.91, 0.89, 0.92, 0.90 + one junk 2.50 → mean 1.170 (wrong), median 0.905 (right). **Why a window:** one 5 cm station may have 0–3 points; ±0.3 m gives ~30.

Re-centring

Left wall at $+w_{\text{left}}$, right wall at $-w_{\text{right}}$ → middle at $(w_{\text{left}} - w_{\text{right}}) / 2$ (positive = move left).
 Example: w_{left} 0.97, w_{right} 0.44 → move +0.26 m. Largest $|w_{\text{left}} - w_{\text{right}}|$: pass 0 = 0.932 m, pass 1 = 0.033 m, pass 2 = 0.011 m.



1 every wall point → its nearest line point, sign = side. 2 one station: median of each side = width. 3 widths from the guide line (dashed) vs final (solid). 4 re-centring shift.

7. validate.py — can we trust the result?

Input

Track + wall points + config

Output

report: 10 checks, each {{passed, severity, value, threshold, note}}

Every check stores **what was measured and the limit**, so track_meta.json alone explains a failure. Example: {"passed": true, "severity": "error", "value": 0.6729, "threshold": 0.2, "note": "smallest of w_left / w_right (m)"}

Check	Severity	Catches
lap_closed	error	lap did not come back to its start (≤ 0.5 m)
min_width	error	a width below 0.2 m
inside_free_space	error	centerline closer than 0.2 m to a wall point (runs through a wall)
gap_fraction_left / right	error	more than 20% of a wall never seen (widths guessed)
length_matches_test_lap	error	centerline length more than 10% off the driven lap
uniform_spacing	error	spacing more than 5% off ds
centerline_simple	error	centerline crosses itself
left / right_wall_folds	warning	wall rebuilt as centerline $\pm$ width loops over itself at a corner sharper than the width (value = s positions). T2/T3/T6 must handle it (inside width $< 1/ \kappa $).

is_valid(report) = all **error** checks passed (warnings do not block). warnings(report) = names of warnings that fired. Deliberately NOT here: curvature (T2's job) and the inside-width $< 1/|\kappa|$ check (T3/T6).

8. export.py — writing the deliverables

Function	What it does
write_track_csv	column_stack([s, xy, w_left, w_right]) → CSV, header s,x,y,w_left,w_right, 4 decimals (0.1 mm)
read_track_csv	the reverse: CSV → N × 5 array (for T2 / plots)
sha256_file	64-character fingerprint of the input log; changes if one byte changes → proves which log made which track
build_meta	assembles track_meta.json
write_outputs	creates the folder, writes both files

```
track_meta.json (synthetic run):
  map_version:  log-sha256:154c87b578d1      (or --map-version <SLAM map ID>)
  frame:        map
  resolution_m:  0.05
  start_pose:    x=6.3, y=0.0, yaw=1.35      (where s = 0 is)
  source_log:    375 frames, 0 skipped lines, sha256, t_start, t_end
  track:         length 32.35 m, 647 points, ds 0.05, closed, direction, units
  stats:         closure error, test-lap length, gap fractions, point counts
  config:        every setting used (reproducible)
  valid, warnings, validation
```

9. pipeline.py and __main__.py — running it

```
build_track(log_path, cfg, out_dir, map_version):
    frames, skipped = read_log(log_path)           # log_io
    acc = accumulate(frames, cfg)                  # accumulate
    track = extract_track(acc.poses, acc.points, cfg) # extract (+ geometry)
    meta = build_meta(track, cfg, validate(...), ...) # validate + export
    write_outputs(out_dir, track, meta)            # export
    return BuildResult(track, accumulated, meta, frames)
```

Command	What it does
<code>python -m t1_track synth out/log.jsonl --points-frame vehicle</code>	fake track + log (+ ground truth csv)
<code>python -m t1_track build out/log.jsonl -o out/track --points-frame vehicle --plot</code>	run T1; --plot writes track.png, track_stages.png (+ track_issues.png if warnings); --ground-truth prints errors; --map-image draws the map underneath
<code>python -m t1_track realtrack <map.yaml> <centerline.csv> -o out/x/log.jsonl</code>	log from a real map with the built-in simulated LiDAR
<code>.venv-gym/Scripts/python tools/gym_record.py Spielberg -o out/gym_spielberg/log.jsonl</code>	log from the official F1TENTH simulator (separate venv)
<code>python -m pytest</code>	all automatic tests (23)

Exit codes of build: **0** valid · **1** built but an error check failed · **2** extraction failed (e.g. incomplete lap).

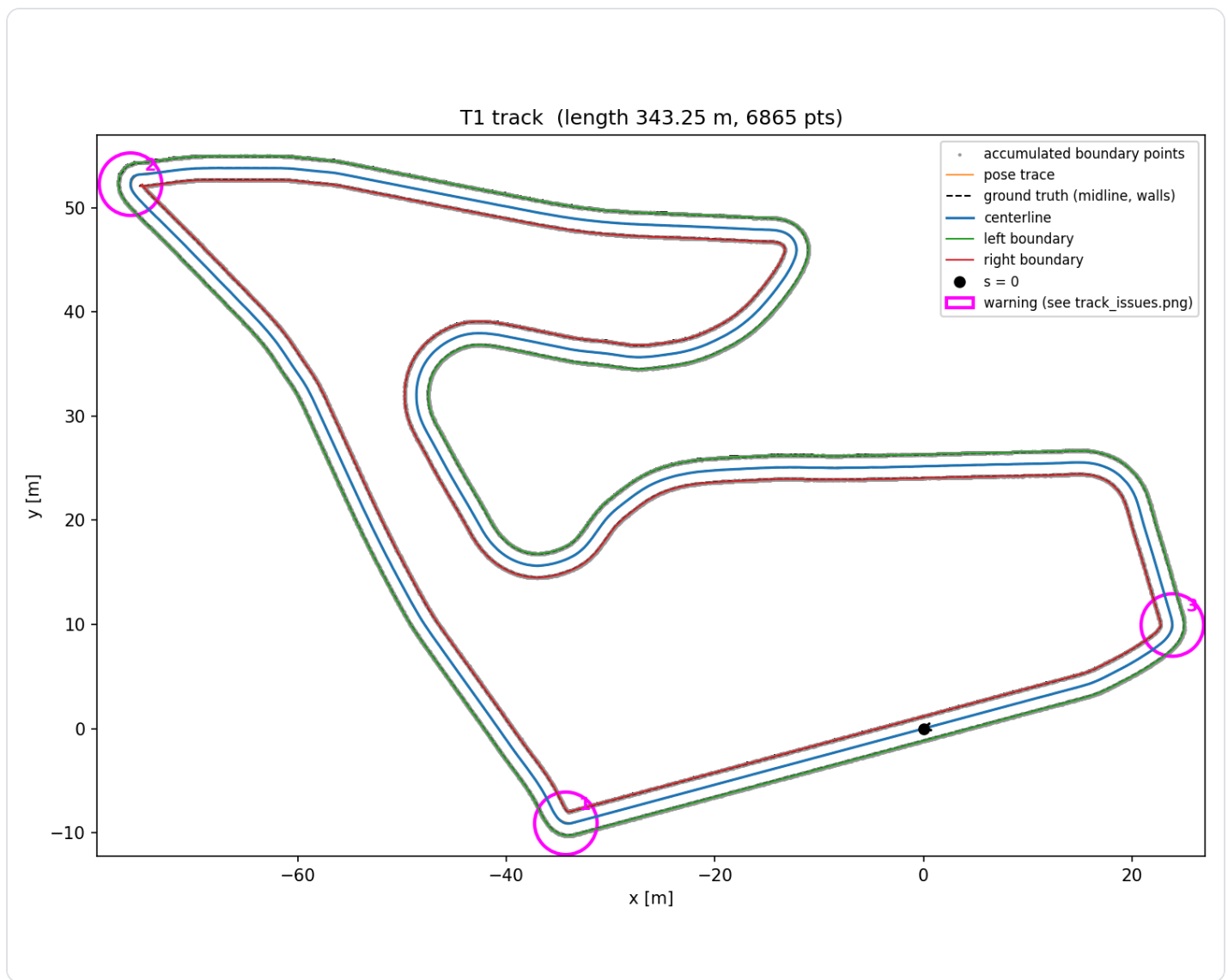
10. How it was tested

Test data	Where from	Centerline error	Width error	Result
synthetic track	synth.py (my generator)	~1–2 cm RMS	~0.5 cm RMS	valid
Spielberg, built-in LiDAR	real 1:10 map + published centerline (f1tenth_racetracks)	0.6 cm RMS	3.4 cm RMS	valid, 2 fold warnings
Spielberg, official simulator	f1tenth_gym v1.0.0 (independent code)	0.5 cm RMS	8.0 cm RMS (sensor/map bias)	valid, 3 fold warnings
Monza, official simulator	f1tenth_gym v1.0.0	0.9 cm RMS	7.2 cm RMS (thick map walls)	valid, 4 fold warnings

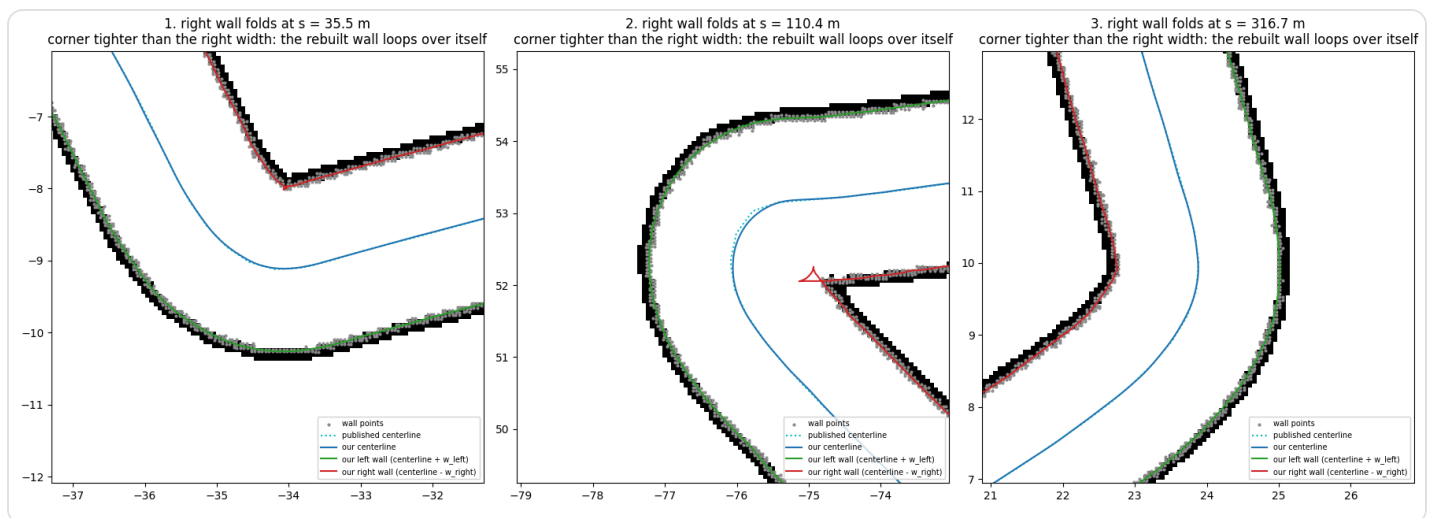
T1 targets (breakdown): centerline RMS ≤ 1 map cell, width RMS ≤ 5 cm. The remaining width bias was measured *independently of T1*: Monza's walls are drawn ~29 cm thick (free space 100 cm/side instead of 110), and the gym's LiDAR itself reads walls +3.6 / +7.1 cm beyond the published width.

Lessons found by testing

- **No-hit LiDAR beams** report $\approx$ max range plus noise (29.99 / 30.01 m) → they became phantom wall points. Fix: the recorder drops beams near max range; T1 ignores points > 10 m. **The future ROS bag → JSON converter must do the same** (range_max, inf, NaN).
- **Convention mismatches** are the typical bug: f1tenth_gym wants [steering, speed] even when configured the other way.
- **Sharp corners** (Spielberg tip, Monza chicane): centerline + width cannot represent them → fold warnings with location. The published data has the same folds.
- Still untested: **real SLAM pose error/drift**, the real base_link → laser offset, real sensor artefacts. Only the first real log answers these.



Spielberg (official simulator): T1 output drawn on the real map. Magenta circles = fold warnings.



Spielberg fold warnings zoomed in: the rebuilt inside wall loops at corners sharper than the 1.1 m width.

11. Open questions for perception

1. Are `boundary_points` in the map frame or relative to the car ($\rightarrow$ `--points-frame`)? If map: SLAM map frame or drifting odom?
2. If relative to the car: relative to which point (`base_link` or the LiDAR)? What is the `base_link` $\rightarrow$ laser offset? (T1 currently assumes 0.)
3. Are scan and pose from the same instant (timestamps)?
4. Conventions: yaw in radians, counter-clockwise; car frame x forward, y left (ROS REP-103)?
5. Is pose raw odometry or SLAM-corrected (loop-closed)?
6. ROS topic names and message types for the recorder (`ros2 bag record` $\rightarrow$ `converter` $\rightarrow$ `log.jsonl`).
7. Physical track: tall walls the 2D LiDAR can see (not floor tape)?

12. Glossary

Word	Meaning
pose	where the car is and which way it faces: (x, y, yaw)
yaw	heading angle in radians, counter-clockwise from the +x axis
frame (coordinate frame)	what the numbers are measured from: the map ('world') or the car ('vehicle')
frame (log frame)	one line of the log: one moment of the test lap
point cloud	a big unordered set of (x, y) points
closed line	points in order where the last connects back to the first
s	distance along the centerline from the start, in the driving direction
station	one point of the line (every 5 cm); widths are computed per station
tangent / normal	unit arrows at a point: forward / 90° to the left
lateral offset	sideways distance of a point from the line (+ left, – right) = dot product with the normal
voxel	one grid square used to merge duplicate points
KD-tree (cKDTree)	data structure for fast 'nearest point' / 'points within r' searches
median	middle value of a sorted list; ignores a few outliers (unlike the mean)
RMS error	root-mean-square: a typical error size, sensitive to large errors
fold (wall fold)	a rebuilt wall looping over itself at a corner sharper than the width

13. Practice files

Each session has a runnable playground with exercises (in out/, not committed): `session1_play.py` (config, log_io) · `session2_play.py` (accumulate) · `session3a_play.py`, `session3b_play.py` (geometry) · `session4_play.py`, `session5_play.py` (extract) · `session6_play.py` (validate, export, pipeline). Run with `python out/session1_play.py`, then uncomment the exercises one at a time and predict the result first.